



Rapport de Projet Drone cible

Partie personnelle
BTS CIEL – option IR

RIBETTE NOLAN

2025-2026

Sebastien GUIVARCH
Denis CAILLE
Francesa BREHONNET



1. Périmètre d'activité.....	3
1.1 Mon rôle dans le projet.....	3
1.2 Liste de mes activités individuelles.....	3
1.3 Diagramme de cas d'utilisation de mon périmètre.....	4
2. Environnement de développement.....	5
2.1 Justification des outils et langages choisis.....	5
2.2 Justification des librairies utilisées.....	6
2.3 Contraintes du client.....	6
3. Architecture du projet.....	7
3.1 Diagramme d'architecture logicielle et matérielle.....	7
3.2 Schéma réseau commenté.....	10
4. Développement – état d'avancement.....	12
4.1 Présentation des composants développés par l'étudiant.....	12
4.2 Extraits de code commentés et expliqués.....	13
4.3 Tableau ou cahier de tests unitaires.....	14
4.4 Gestion des versions.....	15
5. Exploitation réseau et matériels.....	16
5.1 Présentation des équipements.....	16
5.2 Justification des choix matériels.....	17
5.3 Résultats d'un outil d'analyse réseau.....	17
5.4 Gestion cybersécurité.....	19
6. Bilan personnel.....	19
7. Annexes.....	20
Annexe A – Code watchdog radio.....	20
Annexe B – Code navigation verrouillage.....	21
Annexe C – Historique commit github.....	22
Annexe D – Équipement utilisés.....	23

1. Périmètre d'activité

1.1 Mon rôle dans le projet

Au sein de l'équipe, mon rôle a été de développer et mettre en œuvre deux modes critiques du système de pilotage du drone : le mode verrouillage avec détection par caméra, et le mode safemode. Le mode verrouillage permet au drone de maintenir automatiquement une position fixe tout en analysant en temps réel le flux vidéo pour détecter des objets ou des personnes d'intérêt. Le mode safemode constitue le filet de sécurité du système : il regroupe le watchdog radio, qui surveille en permanence la qualité du lien de communication, et le watchdog système, qui supervise l'état général du drone afin de déclencher automatiquement des procédures de sécurité en cas d'anomalie.

1.2 Liste de mes activités individuelles

Dans le cadre de ce projet, mes activités individuelles ont été les suivantes :

Développement du nœud `navigation_verrou` : mise en œuvre du maintien de position automatique via la détection de la caméra, publication des commandes de correction sur le topic ROS2 `/cmd_vel` pour stabiliser le drone sur un point fixe.

Développement du nœud `boat_detector` : récupération du flux vidéo, intégration du pipeline de détection (traitement d'image, inférence du modèle), et publication des résultats de détection sur le topic `/boat_detection_json`.

Développement du nœud `radio_watchdog` : surveillance continue du lien radio (perte de signal), déclenchement automatique du mode arrêt d'urgence en cas de perte de communication dépassant le seuil configuré.

Développement du nœud `system_watchdog` : supervision de l'état système du drone (surveille le bon fonctionnement des nodes), en cas de problème il redémarre le docker concerné par les nodes puis si cela ne fonctionne toujours pas, il notifie l'opérateur du problème.

Tests et validation : vérification du comportement des watchdogs en conditions dégradées simulées (coupure radio, problème node).

Documentation : rédaction des procédures d'activation du mode verrouillage, de configuration des seuils watchdog et des scénarios de bascule automatique vers le safemode.

1.3 Diagramme de cas d'utilisation de mon périmètre

Le diagramme de cas d'utilisation ci-dessous délimite le périmètre fonctionnel des composants dont j'avais la charge, en identifiant les deux acteurs principaux : l'opérateur, qui pilote et supervise le système, et le développeur, qui intervient sur certaines fonctionnalités techniques.

Ce diagramme illustre les interactions entre les acteurs et le système en mode verrouillage. L'opérateur dispose de cinq cas d'utilisation : visualiser le flux vidéo, détecter une cible, valider le verrouillage, et désactiver le verrouillage. La visualisation du flux vidéo est incluse systématiquement dans la détection d'une cible via une relation <<include>>, garantissant que l'opérateur dispose du retour caméra avant toute détection. De même, la validation du verrouillage inclut obligatoirement le chargement vers la cible, assurant ainsi que le drone se positionne effectivement sur la cible validée. Le développeur intervient quant à lui uniquement sur le cas d'utilisation « Charger vers la cible », correspondant à la configuration et à la mise au point du comportement de navigation autonome vers la cible détectée.

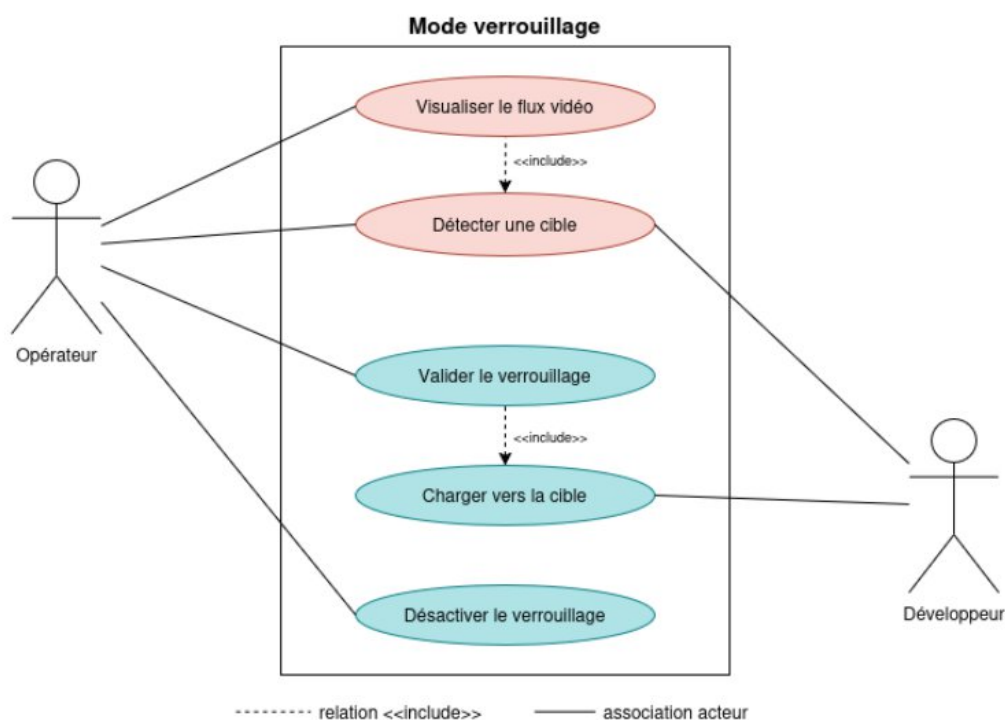


Schéma 1 : Diagramme d'exigence mode verrouillage

Le diagramme de cas d'utilisation ci-dessous délimite le périmètre fonctionnel des composants dont j'avais la charge, en identifiant deux acteurs : le système autonome, qui surveille et réagit sans intervention humaine dans un premier temps, et l'opérateur, qui intervient en dernier recours.

Ce diagramme illustre le fonctionnement du mode Safemode, qui constitue le filet de sécurité du système. Deux watchdogs opèrent en parallèle de manière autonome. Le

watchdog radio surveille en continu la qualité du lien LoRa : en cas de perte de signal dépassant le seuil configuré, il déclenche automatiquement le mode 3 via la publication sur le topic `/robot_mode = 3`, ce qui provoque immédiatement un arrêt d'urgence avec coupure moteur. Le watchdog système surveille quant à lui l'état des nœuds ROS2 : en cas d'anomalie détectée, il tente dans un premier temps de redémarrer automatiquement les nœuds défaillants via une relation `<<include>>`. Si le redémarrage ne suffit pas et que l'anomalie persiste, le watchdog système notifie alors l'opérateur via l'interface de supervision, grâce à une relation `<<extend>>`, lui permettant de prendre la main et d'intervenir manuellement.

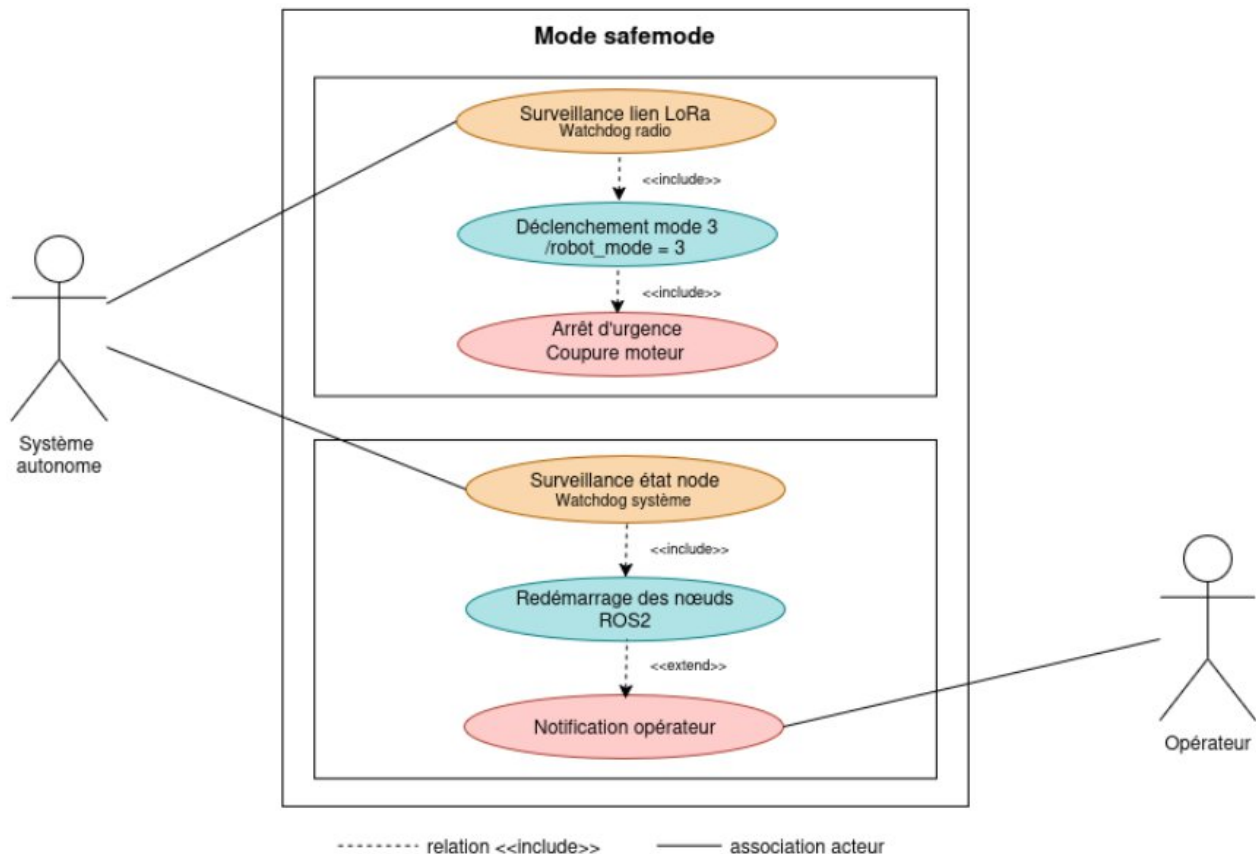


Schéma 2 : Diagramme d'exigence mode safemode

2. Environnement de développement

2.1 Justification des outils et langages choisis

Dans le cadre de ce projet, le choix des outils a été guidé par des contraintes de performance, de compatibilité embarquée et de facilité de développement.

Le langage Python 3 a été retenu car il permet un développement rapide et une grande lisibilité du code. Il est particulièrement adapté à ROS2 grâce à la bibliothèque `rclpy`, qui facilite la création de nœuds et la gestion des communications. De plus, Python est largement utilisé dans les domaines de la robotique et de l'intelligence artificielle.

Le framework ROS2 constitue le cœur de l'architecture logicielle. Il permet de structurer le système sous forme de nœuds indépendants communiquant entre eux via des topics. Ce modèle favorise la modularité, la maintenabilité et la réutilisation du code. ROS2 est également conçu pour des systèmes distribués et temps réel, ce qui correspond aux besoins du projet.

L'utilisation de Docker permet de garantir un environnement de développement identique entre les différentes machines (poste de développement et Raspberry Pi). Cela évite les problèmes de dépendances et simplifie le déploiement du projet. De plus ros2 est compatible sur docker.

Enfin, GitHub est utilisé pour le versionnement du code. Il permet de suivre l'évolution du projet, de conserver un historique des modifications et de structurer le travail en différentes versions.

2.2 Justification des librairies utilisées

Plusieurs librairies ont été intégrées afin de répondre aux besoins spécifiques du projet.

La bibliothèque rclpy permet de créer et gérer les nœuds ROS2 en Python. Les packages std_msgs et sensor_msgs sont utilisés pour définir les formats de communication entre les différents composants.

Pour la partie vision, la bibliothèque OpenCV (cv2) est utilisée pour traiter les images issues de la caméra. Elle permet de manipuler les flux vidéo en temps réel.

Le framework PyTorch (torch) est utilisé pour exécuter le modèle de détection YOLOv5, permettant d'identifier des objets (bateaux) dans les images.

Enfin, cv_bridge assure la conversion entre les formats d'images ROS et OpenCV, ce qui est indispensable pour intégrer la vision dans ROS2.

2.3 Contraintes du client

Le projet doit répondre à plusieurs contraintes.

Sur le plan matériel, le système repose sur un Raspberry Pi 5, qui possède des ressources limitées en calcul et en mémoire. Il est donc nécessaire d'optimiser les traitements, notamment pour la vision par ordinateur.

Sur le plan fonctionnel, le robot doit être capable de réagir rapidement à son environnement, ce qui impose des contraintes de temps réel.

Enfin, des contraintes de sécurité sont présentes : le robot doit être capable de s'arrêter automatiquement en cas de perte de communication ou de défaillance d'un composant. Cela justifie la mise en place d'un système de watchdog.

3. Architecture du projet

3.1 Diagramme d'architecture logicielle et matérielle

L'architecture du projet repose sur une organisation modulaire en plusieurs nœuds ROS2, chacun ayant un rôle spécifique :

Le nœud USB_cam récupère les images de la caméra pour les publier sur le topic /image_raw. Les informations sont ensuite transmises au nœud boat_detector ainsi qu'au pc poste pilotage.

Le nœud Boat Detector récupère les images de la caméra et détecte les objets à l'aide de YOLOv5. Les informations sont ensuite transmises au nœud de Navigation avec le topic /boat_detection_json qui fait apparaître les coordonnées du rectangle où a eu lieu la détection puis la hauteur et largeur du rectangle.

Le nœud Navigation_verrou permet de faire fonctionner les moteurs en fonction de si il y a eu une détection d'un bateau et de le suivre. Ce nœud s'active quand /robot_mode passe à 2 ce qui signifie que le mode verrouillage est activée.

Le nœud Robot Controller permet de contrôler les moteurs avec le topic /cmd_vel, ce topic publie les données des axes X, Y, Z. Ce nœud s'adapte donc aux commandes envoyées puis fait fonctionner les moteurs en fonction des requêtes.

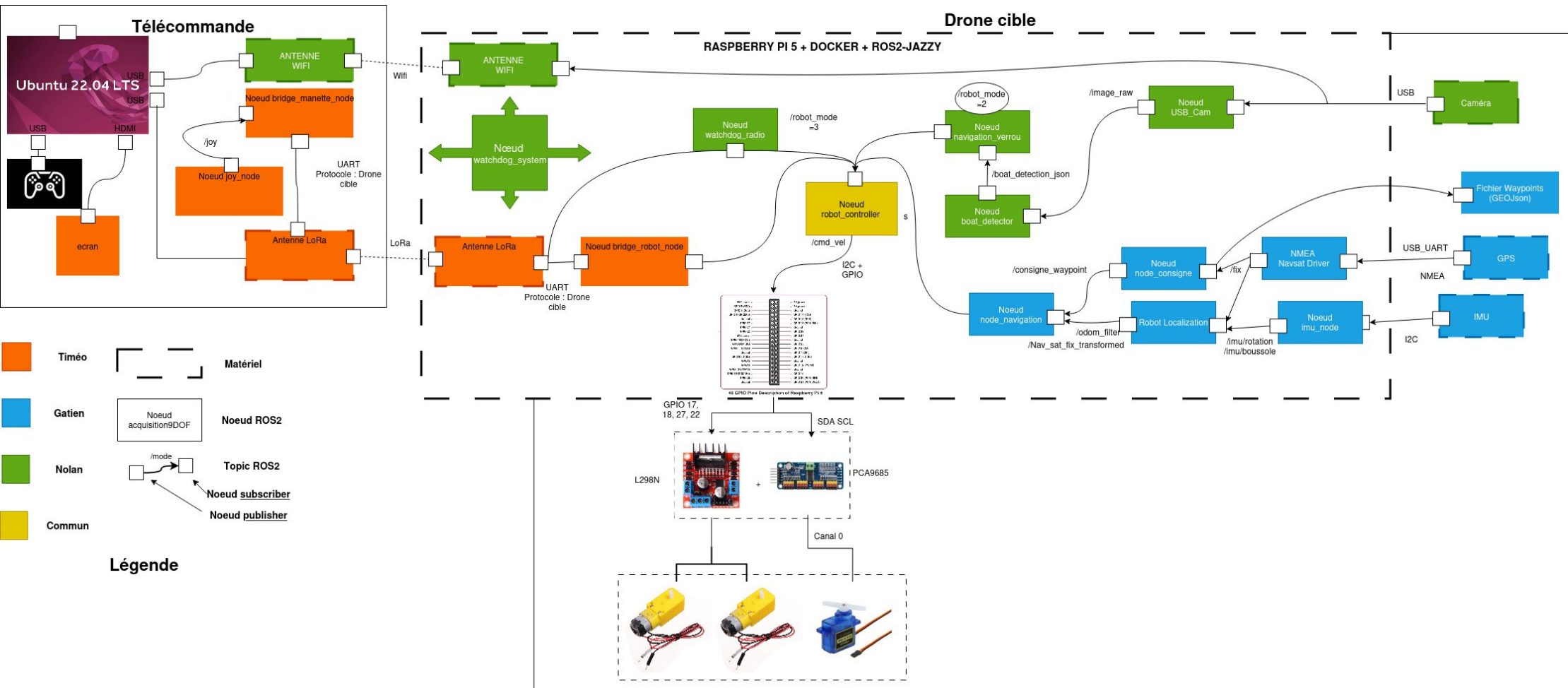


Schéma 3 : Diagramme global projet drone cible

Le noeud Watchdog_radio est abonné au topic `bridge_drone` et surveille continuellement la réception des messages. À chaque message reçu, un temporisateur interne est réinitialisé. Tant que les messages arrivent à intervalle régulier, le watchdog considère que la liaison LoRa est valide et le robot continue de fonctionner normalement.

En revanche, si aucun message n'est reçu pendant une durée définie (par exemple plusieurs secondes), le noeud `watchdog_radio` considère qu'il y a une perte de communication radio. Cette situation peut correspondre à :

- une coupure du lien LoRa,
- un arrêt du bridge radio,
- un crash logiciel,
- ou une perte d'alimentation de la télécommande.

Lorsque cette perte de communication est détectée, le noeud `Watchdog_radio` publie automatiquement une valeur de sécurité sur le topic `/robot_mode`.

Le mode envoyé est :

- Mode = 3

Ce mode correspond à un état d'arrêt d'urgence ou de sécurité. Les autres noeuds du robot, notamment le contrôleur moteur (`robot_controller`), surveillent également le topic `/robot_mode`. Lorsqu'ils détectent le passage en mode 3, ils coupent immédiatement les moteurs afin d'empêcher tout mouvement incontrôlé du robot.

Ce système permet donc d'ajouter une couche de sécurité essentielle au robot en garantissant qu'en cas de perte du lien radio, le robot s'arrête automatiquement sans intervention humaine.

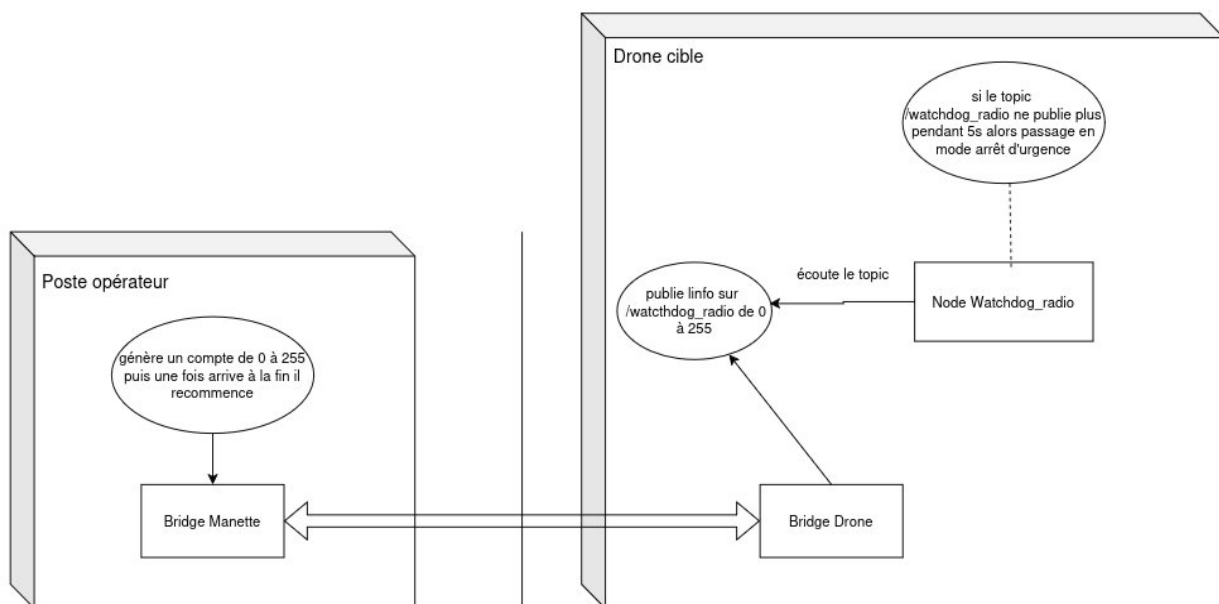


Schéma 4 : Fonctionnement watchdog radio

Enfin, le nœud `watchdog_system` constitue le cœur du système de surveillance. Il s'agit d'un watchdog système dont le rôle est de surveiller en permanence le bon fonctionnement des autres nœuds ROS2 (`boat_detector`, `bridge_controller` ou `robot_controller`) s'exécutant sur le Raspberry Pi. Comme illustré dans le schéma ci-dessous, son fonctionnement repose sur une logique en deux étapes : dans un premier temps, si un nœud est détecté comme défaillant, le watchdog tente automatiquement de relancer le conteneur Docker correspondant afin de le récupérer. Si, après ce redémarrage, le nœud ne parvient toujours pas à fonctionner correctement, le système bascule vers une alerte : le nœud `watchdog_system` publie alors sur le topic `/error_node` avec la valeur `data="66"`, signalant à l'opérateur qu'une intervention humaine est nécessaire. Cette approche garantit une résilience maximale tout en évitant que le robot reste bloqué silencieusement en cas de panne.

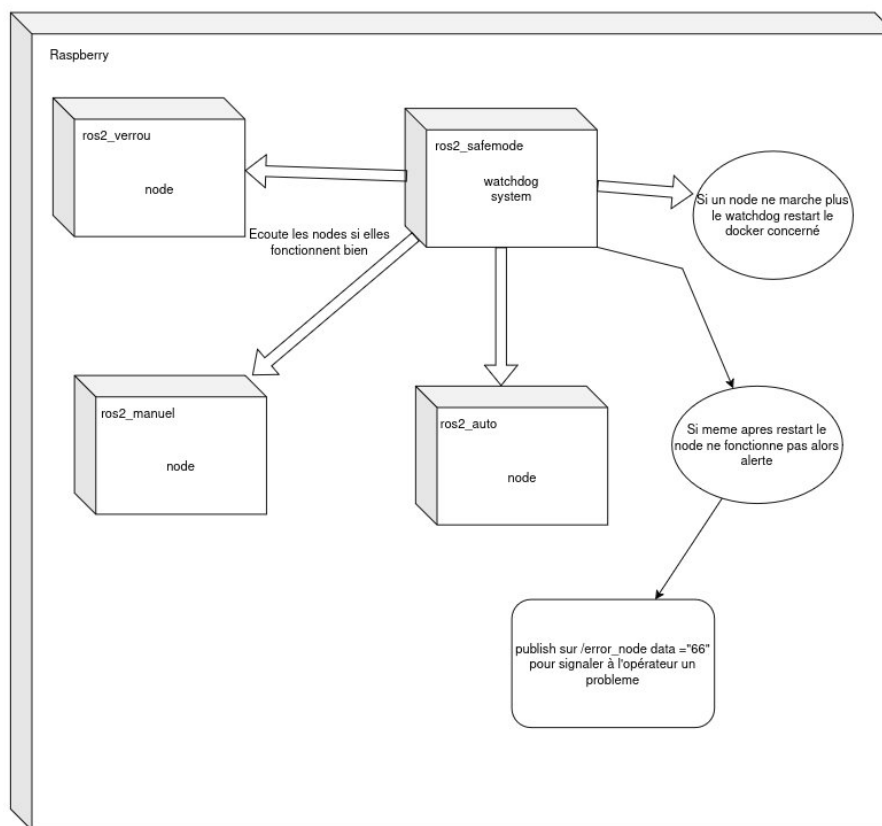


Schéma 5 : Fonctionnement watchdog système

3.2 Schéma réseau commenté

Le système est composé de deux entités principales : un poste de développement et un robot embarqué.

PC (développement) --- SSH ---> Raspberry Pi (ROS2 + Docker)

Le Raspberry Pi exécute les nœuds ROS2 à l'intérieur d'un conteneur Docker. Le poste de développement permet de déployer le code, surveiller les logs et interagir avec le système via des commandes ROS2.

Les communications internes entre nœuds utilisent le protocole DDS, intégré à ROS2, qui permet des échanges rapides et fiables. (je précise que pour le moment je surveille Publisher test et non bridge_operator pour des raisons en d'attente du camarade qui le développe).

Cette architecture réseau concerne la liaison Wi-Fi quand le projet est en maintenance / développement. Le Wi-Fi utiliser est celui du lycée qui nous permet de faire fonctionner notre système sans antenne pour effectuer des tests.

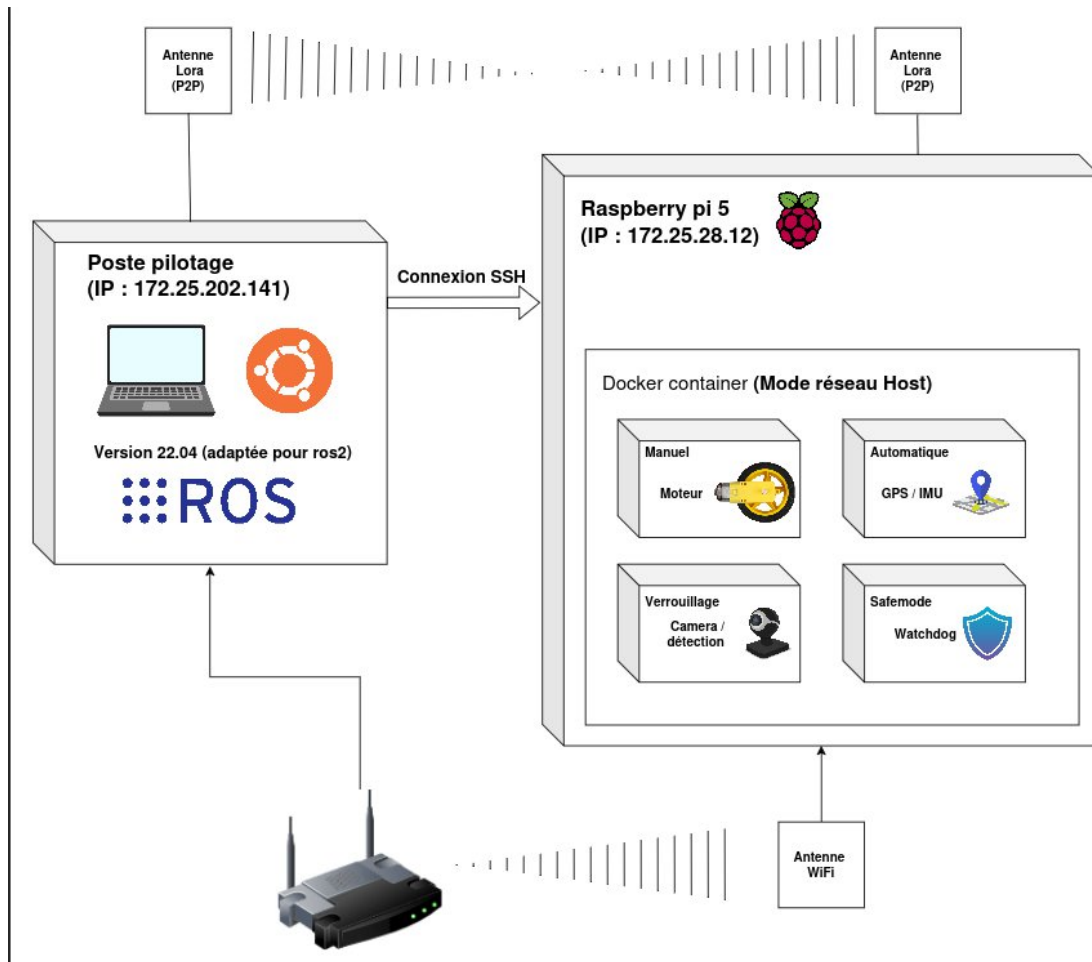


Schéma 6 : Schéma réseau en mode maintenance / développement

Ma contribution spécifique dans cette architecture réseau concerne la liaison Wi-Fi longue portée destinée au retour vidéo en temps réel. Le drone embarque une antenne Wi-Fi émettrice connectée au Raspberry Pi quand il est en fonctionnement, qui diffuse en continu le flux de la webcam via le réseau local sans fil. De l'autre côté, une antenne Wi-Fi réceptrice est installée sur le poste opérateur (un autre bateau ou la station au sol), qui reçoit ce flux et l'affiche en direct. Ce dispositif permet à l'opérateur de visualiser ce que "voit" le drone à tout moment, même lorsque celui-ci s'éloigne, grâce au gain des antennes qui assure une portée et une stabilité bien supérieures au Wi-Fi standard. Ce retour vidéo est également indispensable pour la supervision de la détection YOLOv5 : l'opérateur peut ainsi vérifier en temps réel que les cibles (bateaux) sont correctement identifiées par le système.

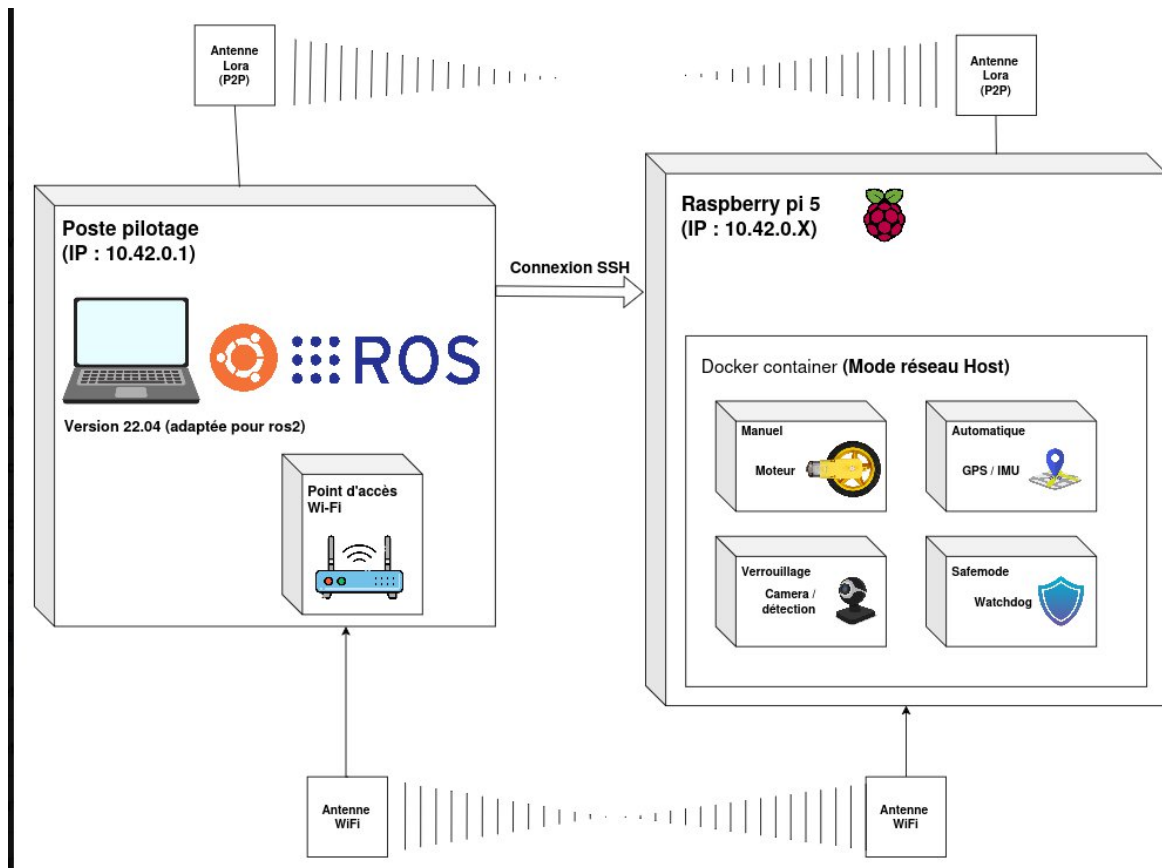


Schéma 7: réseau en mode maintenance / développement

4. Développement – état d'avancement

4.1 Présentation des composants développés par l'étudiant

Le traitement sur le Raspberry Pi (colonne centrale)

C'est le cerveau du drone. Les données traversent 4 étapes dans l'ordre :

1. `usb_cam_node` reçoit les images de la webcam et les diffuse sur le réseau interne ROS2 via le topic `/image_raw`.
2. `boat_detector` reçoit ces images et les analyse avec YOLOv5 pour repérer les bateaux. Quand il en détecte un, il publie ses coordonnées (position, taille du rectangle) sur le topic `/boat_detection_json`.
3. `ros2_auto` ou `ros2_manuel` reçoivent ces coordonnées. En mode auto, il calcule automatiquement la direction à prendre pour suivre le bateau. En mode manuel, c'est l'opérateur qui pilote. Les deux envoient leurs ordres via `/cmd_vel`.
4. `robot_controller` reçoit les ordres `/cmd_vel` et les traduit en signaux concrets pour faire tourner les moteurs.

Le `ros2_safemode` (en rouge) surveille tout ça en parallèle. Si un nœud tombe en panne, il tente de le relancer. Si ça ne marche toujours pas, il coupe les moteurs et envoie une alerte `/error_node = 66`.

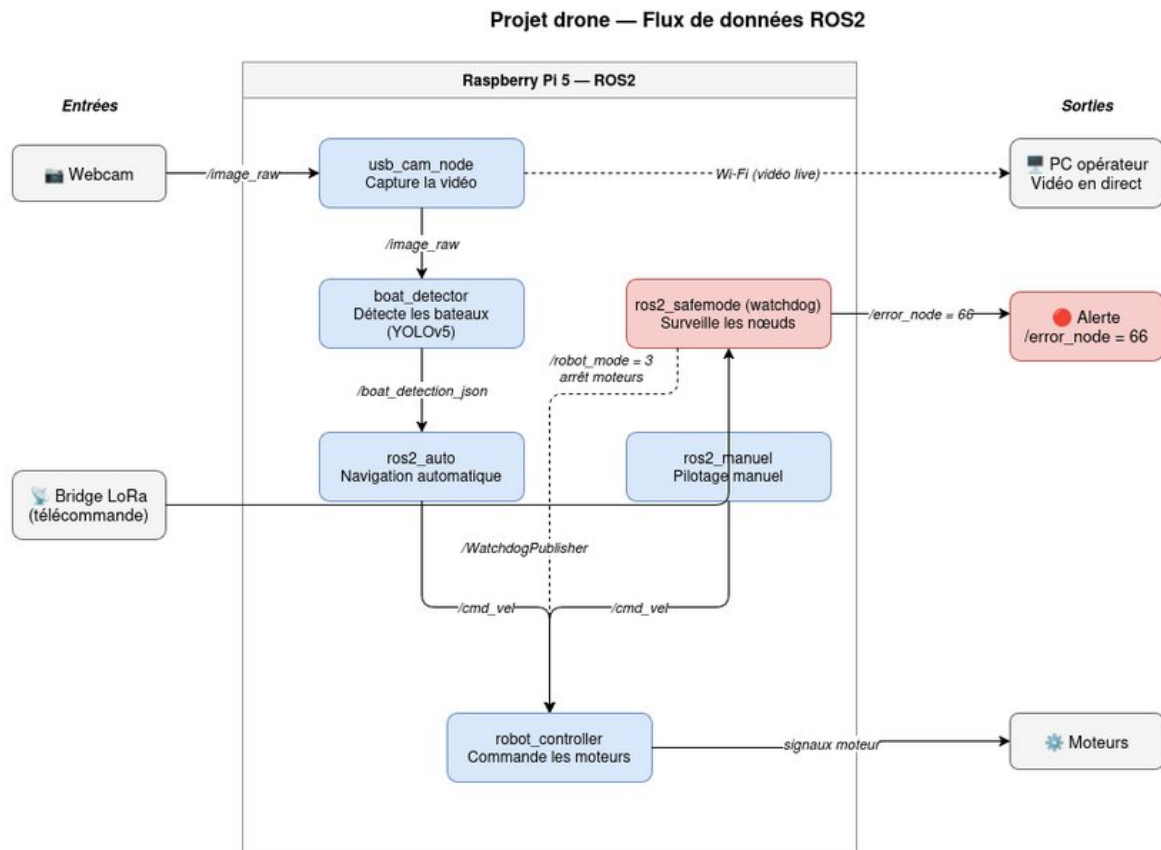


Schéma 8 : Diagramme de données

4.2 Extraits de code commentés et expliqués

Cette section présente deux extraits représentatifs du code développé : le mécanisme de surveillance radio (watchdog) et la boucle de navigation en mode verrouillage.

Extrait 1 — Watchdog radio (nœud radio_watchdog)

Le watchdog radio surveille en permanence la réception de messages sur le topic `/bridge_drone`. Afin de prévenir tout comportement incontrôlé en cas de perte du lien LoRa ou de défaillance logicielle, un mécanisme de « battement de cœur » (heartbeat) a été mis en place : si aucun message n'est reçu pendant plus de 3 secondes, le système bascule automatiquement en mode sécurité.

Voir [annexe](#) Code watchdog radio

Le fonctionnement repose sur trois étapes :

- **Mesure du délai** : à chaque appel, la méthode calcule le temps écoulé depuis la dernière réception de message (variable `delta`).
- **Déclenchement de la sécurité** : si `delta` dépasse 3 secondes, un message de valeur "3" est publié sur le topic `/robot_mode`, ce qui force l'arrêt immédiat des moteurs par le nœud `robot_controller`.

- **Réinitialisation** : le compteur est remis à zéro après chaque déclenchement afin d'éviter une boucle d'alertes infinies.

Extrait 2 — Navigation en mode verrouillage (nœud NavigationNode)

Le nœud NavigationNode assure le suivi automatique de la cible détectée par YOLOv5. La boucle de contrôle tourne à 20 Hz et applique un correcteur proportionnel (P) pour maintenir le bateau centré dans le champ de la caméra.

Voir [annexe](#) Code navigation verrouillage

La logique de commande s'articule en quatre étapes :

- **Vérification du mode actif** : si `robot_mode` $\neq$ 2, aucune commande n'est envoyée et le drone reste immobile, assurant ainsi une sécurité de base.
- **Récupération de la cible** : les coordonnées `x_min` et `width` de la bounding box fournie par YOLOv5 permettent de calculer le centre horizontal du bateau détecté.
- **Calcul de l'erreur et loi de commande** : l'écart entre le centre du bateau et le centre de l'image (320 px) est multiplié par le gain `Kp` (0,002) pour produire une vitesse angulaire corrective. Une vitesse linéaire constante de 0,6 m/s est appliquée en parallèle pour avancer vers la cible.
- **Zone morte** : une tolérance de 20 pixels évite que les moteurs forcent inutilement lorsque le bateau est déjà quasiment centré, réduisant ainsi les oscillations de trajectoire.

4.3 Tableau ou cahier de tests unitaires

Afin de valider le bon fonctionnement des composants développés, cinq tests unitaires ont été réalisés, couvrant les nœuds critiques du système.

Le premier test a porté sur le watchdog radio : il vérifie que le drone bascule automatiquement en mode sécurité (mode 3) après 3 secondes sans réception de message sur le topic `/robot_mode`. Pour simuler le nœud `bridge_operator`, un script Python nommé `PublisherWatchdog` a été développé. Celui-ci publie des valeurs à intervalle d'une seconde en démarrant à 1000, et déclenche une simulation de crash à la valeur 1050, ce qui provoque la réaction attendue du watchdog.

Le deuxième test a validé le watchdog système, en vérifiant que le redémarrage automatique du conteneur Docker est bien effectué lorsqu'un nœud ROS2 tombe en défaillance.

Les tests trois, quatre et cinq ont porté sur la détection de cibles via le nœud `boat_detector` ainsi que sur le mode verrouillage du nœud `navigation_verrou`. Ils ont permis de confirmer que les coordonnées JSON sont correctement publiées lors d'une détection, que le drone suit bien la cible lorsque le mode 2 est actif, et qu'il reste immobile dans le cas contraire. Des améliorations restent possibles sur la précision du modèle de détection YOLOv5.

L'ensemble de ces tests s'est conclu avec succès, tous les résultats obtenus étant conformes aux résultats attendus.

Tests Unitaires – Drone Cible (Nolan RIBETTE)						
ID	Composant testé	Objectif du test	Étapes / Données d'entrée	Résultat attendu	Résultat obtenu	Statut
TU-01	watchdog_radio (radio_watchdog.py)	Vérifier que le drone bateau passe en mode sécurité (mode 3) après 3 s sans signal radio.	1. Lancer le nœud watchdog_radio 2. Stopper la publication sur /bridge_drone 3. Attendre 3 secondes	Le topic /robot_mode publie la valeur "3" Les moteurs s'arrêtent	Le topic /robot_mode publie la valeur "3" Les moteurs s'arrêtent	OK
TU-02	watchdog_system (system_watchdog.py)	Vérifier que le watchdog système redémarre un conteneur Docker si un nœud ROS2 tombe.	1. Lancer watchdog_system 2. Tuer manuellement le nœud boat_detector 3. Observer le comportement	Le conteneur Docker est relancé automatiquement Le nœud reprend en < 10 s	Le conteneur Docker est relancé automatiquement Le nœud reprend en < 10 s	OK
TU-03	boat_detector (YOLOv5 + OpenCV)	Vérifier que la détection d'une cible bateau publie les bonnes coordonnées JSON.	1. Publier une image avec une cible bateau sur /image_raw 2. Lire le topic /boat_detection_json	La cible bateau est bien détecté par le nœud, le JSON contient x_min, width, height	Affichage du JSON qui contient x_min, width, height	OK
TU-04	navigation_verrou (NavigationNode)	Vérifier que le drone bateau soit en mouvement lors d'une cible détectée en mode verrouillage (robot_mode = 2).	1. Activer robot_mode = 2 2. Publier une détection 3. Lire /cmd_vel	La valeur /cmd_vel varie et le drone bateau est en mouvement	Variation de /cmd_vel et mouvement du drone bateau quand une cible bateau est détecté	OK
TU-05	navigation_verrou (NavigationNode)	Vérifier que le drone bateau s'arrête si robot_mode ≠ 2 (mode verrouillage inactif).	1. Laisser robot_mode = 0 (ou 1) 2. Publier une détection bateau 3. Analyser si la détection fait bouger le drone bateau	Le drone bateau n'est pas censé bouger car le mode est ≠ 2	Le drone bateau ne réalise aucun mouvement même si une cible bateau est détecté	OK

4.4 Gestion des versions

Le code source est versionné sur GitHub, permettant un suivi précis des modifications via l'historique des commits et un retour aisé à une version antérieure en cas de problème.

L'ensemble du code source du projet est versionné sur un dépôt GitHub privé (guivarchVauban / Drone_cible_2026_repo). Cet outil a été utilisé tout au long du développement pour assurer la traçabilité des modifications, faciliter les retours en arrière en cas de régression et organiser le travail par fonctionnalité.

Historique des commits

La capture ci-dessous présente l'historique des commits sur la branche main. On y distingue trois grandes périodes de développement : la mise en place de la détection YOLOv5 (janvier 2026), la configuration de l'environnement Docker (février 2026) et la documentation du projet.

Voir [annexe](#) de l'historique des commits github

Synthèse des commits clés

Date	Commit	Hash
13 fév. 2026	docker-compose-camera + Création du Dockerfile	7b0c550
13 fév. 2026	Renommage yolo_node.py → yolo_node_affichage.py	6c7fbc2
12 fév. 2026	Create README.md	c9cc89f
12 fév. 2026	Création et configuration du nœud YOLOv5	12ef66d
12 fév. 2026	Setup v1 nœud YOLOv5 (Setup_v1_node_yolov5)	cdcbaab
7 jan. 2026	Add section on technologies used in the project	8b16ace
7 jan. 2026	Add description for visual detection mode (README)	530d17e
7 jan. 2026	Merge branch 'main' — fusion des branches	9182a22

Tableau 1 — Sélection des commits les plus significatifs du projet

Bonnes pratiques appliquées

- **Messages de commit explicites** : chaque commit décrit précisément la modification apportée, facilitant la relecture de l'historique.
- **Utilisation des branches** : une branche dédiée a été utilisée pour les développements en cours avant fusion sur main via des merge commits.
- **Traçabilité complète** : chaque hash de commit permet de revenir à n'importe quel état du projet, garantissant une sécurité en cas de régression.

5. Exploitation réseau et matériels

5.1 Présentation des équipements

Le système embarqué repose sur les composants suivants [annexe](#) équipements utilisés.

Raspberry Pi 5 : cœur du système, elle exécute ROS 2, le modèle de détection YOLOv5 et la logique de navigation. La webcam USB lui fournit le flux vidéo analysé en temps réel. Les deux antennes Wi-Fi à gain assurent le retour vidéo longue portée et maintiennent le lien radio nécessaire au watchdog.

PCA9685 & L298N : le contrôleur PWM gère le servomoteur (gouvernail) et la vitesse des moteurs avec une précision de 12 bits via I2C. Le pont en H L298N fournit la puissance nécessaire aux deux moteurs DC tout en isolant électriquement la Raspberry Pi des pics de tension.

5.2 Justification des choix matériels

La Raspberry Pi 5 a été retenue pour sa capacité à exécuter YOLOv5 et ROS 2 simultanément avec une latence minimale, indispensable pour l'asservissement en temps réel. L'architecture PCA9685 + L298N permet une commande PWM précise et protège la carte de tout retour de courant. Les antennes Wi-Fi à gain compensent les limites du Wi-Fi standard en milieu aquatique, aussi bien pour le streaming vidéo que pour le maintien du lien watchdog. Enfin, le servomoteur offre une direction plus précise qu'une approche différentielle, ce qui améliore la stabilité du drone à haute vitesse.

5.3 Résultats d'un outil d'analyse réseau

Quatre tests de capture Wireshark ont été réalisés afin de mesurer le débit réel généré par le flux vidéo transmis via ROS 2 / CycloneDDS entre la Raspberry Pi et le poste opérateur. Les tests font varier deux paramètres : la résolution (640×480 et 320×240) et la fréquence d'images (30 FPS et 15 FPS).

Tableau de volumétrie vidéo — ROS 2 / CycloneDDS

ID	Résolution	FPS	Débit moy. (Mbit/s)	Paquets/s	Taille moy. paquet (o)	Part trafic	Volume ~30s	Volume 3 min
#1	640×480	30	162	2 465	8 218	99,9 %	607,5 Mo	3,64 Go
#2	640×480	15	81	1 293	8 767	99,9 %	303,8 Mo	1,82 Go
#3	320×240	30	37	361	12 935	99,6 %	138,8 Mo	0,83 Go
#4	320×240	15	19	183	12 946	99,4 %	71,2 Mo	0,43 Go

Tableau 2 — Synthèse des captures Wireshark (données réelles, projections calculées)

Analyse des résultats :

Dominance du flux vidéo : dans tous les tests, le flux vidéo représente entre 99,4 % et 99,9 % du trafic réseau total. Les autres échanges ROS 2 sont négligeables.

Impact de la résolution : passer de 640×480 à 320×240 à 30 FPS divise le débit par 4,4 (162 → 37 Mbit/s), sans nécessairement compromettre la détection YOLOv5.

Impact du framerate : réduire de 30 à 15 FPS divise le débit par 2. À 640×480 / 15 FPS, le débit tombe à 81 Mbit/s, compatible avec la liaison Wi-Fi longue portée.

Taille des paquets : les paquets en 320×240 sont ≈ 4 à 5× plus grands (12 900 o) qu'en 640×480 (8 200 o), dû à la fragmentation différente des trames UDP/DDS.

```
4 |
▶ Frame 20: 13578 bytes on wire (108624 bits), 13578 bytes captured (108624 bits)
▶ Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
▶ Internet Protocol Version 4, Src: 172.25.28.12, Dst: 172.25.28.12
▶ User Datagram Protocol, Src Port: 56584, Dst Port: 20421
▶ Real-Time Publish-Subscribe Wire Protocol
```

Image - Capture d'une trame RTPS du nœud image_raw publiant sur le réseau

- La trame Wireshark montre une communication **locale au Raspberry Pi** entre deux nœuds ROS 2 (pas de réseau physique impliqué).
- **Couche 2 — Ethernet II** : adresses MAC à zéro → interface loopback, la trame ne quitte pas la machine.
- **Couche 3 — IPv4** : IP source = IP destination (172.25.28.12) → confirme la communication interne, typique d'une architecture Docker sur le même hôte.
- **Couche 4 — UDP** : UDP est choisi par CycloneDDS car il évite les délais de retransmission de TCP. Pour un flux vidéo temps réel, la latence prime sur la fiabilité.
- **Couche 7 — DDS/RTPS** : protocole applicatif de ROS 2. Les 13 578 octets correspondent à un fragment d'image du topic /image_raw, ce qui explique les 99,9 % du trafic mesurés dans les tests Wireshark.
- En opération réelle, la structure est identique — seules les adresses MAC et IP changent lorsque les paquets transitent par le Wi-Fi.

Analyse OSI — Trame Wireshark · Frame 20 · 13 578 octets

	Protocole	Nom Wireshark	Informations clés
Application	DDS / RTPS	Real-Time Publish-Subscribe Wire Protocol	Protocole ROS 2 / CycloneDDS Transport des messages entre nœuds Flux vidéo, topics /cmd_vel, /robot_mode
Transport	UDP	User Datagram Protocol	Port source : 56 584 Port destination : 20 421 Sans connexion — faible latence (temps réel)
Réseau	IPv4	Internet Protocol Version 4	Src : 172.25.28.12 Dst : 172.25.28.12 (même adresse) Communication locale sur le Raspberry Pi
Liaison	Ethernet II	Ethernet II	Src MAC : 00:00:00:00:00:00 Dst MAC : 00:00:00:00:00:00 Interface loopback (pas de réseau physique)

Tableau 3 - Couche modèle OSI de la trame wireshark

5.4 Gestion cybersécurité

Plusieurs mesures de sécurité ont été mises en place :

- **Docker** : les nœuds ROS 2 s'exécutent dans des conteneurs isolés, réduisant la surface d'attaque et les risques de mauvaise configuration.
- **SSH par clé** : la connexion au Raspberry Pi est sécurisée par une paire de clés cryptographiques (clé publique dans `authorized_keys`, clé privée sur le poste de développement), éliminant les risques liés aux mots de passe et aux attaques par force brute.
- **Watchdog** : la surveillance continue du lien radio et des nœuds ROS 2 garantit l'arrêt automatique des moteurs en cas de perte de communication ou de défaillance logicielle.

6. Bilan personnel

Ce projet de drone cible d'entraînement pour la Marine nationale m'a permis d'acquérir de nouvelles compétences techniques et de consolider celles déjà acquises durant la formation. J'ai principalement travaillé sur les fonctions critiques du système, notamment le mode verrouillage par détection caméra ainsi que le système de sécurité Safemode basé sur les watchdogs radio et système.

La mise en place des nœuds ROS2 m'a permis d'approfondir le fonctionnement d'une architecture robotique distribuée et d'améliorer ma maîtrise de ROS2, des topics, des publications et abonnements ainsi que de la communication entre composants logiciels. Le développement des watchdogs m'a également sensibilisé aux problématiques de sûreté de fonctionnement et de résilience des systèmes embarqués, avec la gestion des pannes, le redémarrage automatique des services Docker et la mise en sécurité du robot.

La partie vision embarquée avec YOLOv5 a constitué un autre apport important. J'ai découvert l'intégration d'un modèle d'intelligence artificielle dans un environnement contraint comme le Raspberry Pi, ce qui m'a amené à prendre en compte les limites matérielles et les problématiques d'optimisation.

Ce projet m'a aussi appris à travailler en équipe sur un développement commun avec GitHub, à documenter mes réalisations et à réaliser des tests afin de valider les fonctionnalités développées. J'ai également dû adapter certaines parties du projet en fonction de l'avancement des autres membres, ce qui m'a demandé de faire preuve d'autonomie et d'adaptation.

Enfin, malgré les résultats obtenus, certaines pistes d'amélioration restent envisageables, notamment l'optimisation de la précision du suivi automatique, l'amélioration du modèle de détection YOLOv5 ainsi que la finalisation des tests réseau en conditions réelles. Ce projet a été très formateur car il m'a permis de mettre en pratique plusieurs domaines étudiés durant le BTS : robotique, réseau, systèmes embarqués, cybersécurité et développement logiciel.

7. Annexes

Annexe A – Code watchdog radio

```
def check_watchdog(self):  
    """  
    Vérifie périodiquement l'état de santé de la communication.  
    Si aucun message n'est reçu pendant plus de 3 secondes,  
    une procédure de mise en sécurité est déclenchée.  
    """  
  
    # Calcul du temps écoulé depuis la dernière réception  
    delta = (datetime.now() - self.last_msg_time).total_seconds()  
  
    if delta > 3.0:  
        self.get_logger().warn(f"Alerte : Perte de signal ({delta:.2f}s)")  
  
        # Création du message d'arrêt d'urgence  
        msg = String()  
        msg.data = "3" # mode 3 = arrêt complet des moteurs  
  
        # Publication de l'ordre de coupure vers les actionneurs  
        self.mode_publisher.publish(msg)  
  
        # Réinitialisation pour éviter une boucle d'alerte infinie  
        self.last_msg_time = datetime.now()
```

Figure 1 — Méthode `check_watchdog()` du nœud `radio_watchdog`

Annexe B – Code navigation verrouillage

```
def control_loop(self):
    """
    Boucle de contrôle (20 Hz) : calcule les commandes moteurs
    pour centrer le drone sur le bateau détecté.
    """
    twist = Twist()

    # Sécurité : si le mode verrou (2) n'est pas actif, on stoppe tout
    if not self.locked or not self.last_detection:
        self.cmd_pub.publish(twist)
        return

    # Extraction des coordonnées du bateau détecté par YOLOv5
    boat = self.last_detection["boats"][0]
    center_x = float(boat.get("x_min", 0.0)) + (float(boat.get("width", 0.0)) /
    2.0)

    # Calcul de l'erreur : écart entre le centre du bateau et le centre image
    error = center_x - self.image_center_x

    Kp = 0.002          # Gain proportionnel
    dead_zone = 20.0     # Zone morte en pixels

    twist.linear.x = 0.6      # Vitesse d'avance constante

    if abs(error) > dead_zone:
        twist.angular.z = float(-Kp * error) # Correction angulaire

    self.cmd_pub.publish(twist)
```

Figure 2 — Méthode control_loop() du nœud NavigationNode

Annexe C – Historique commit github

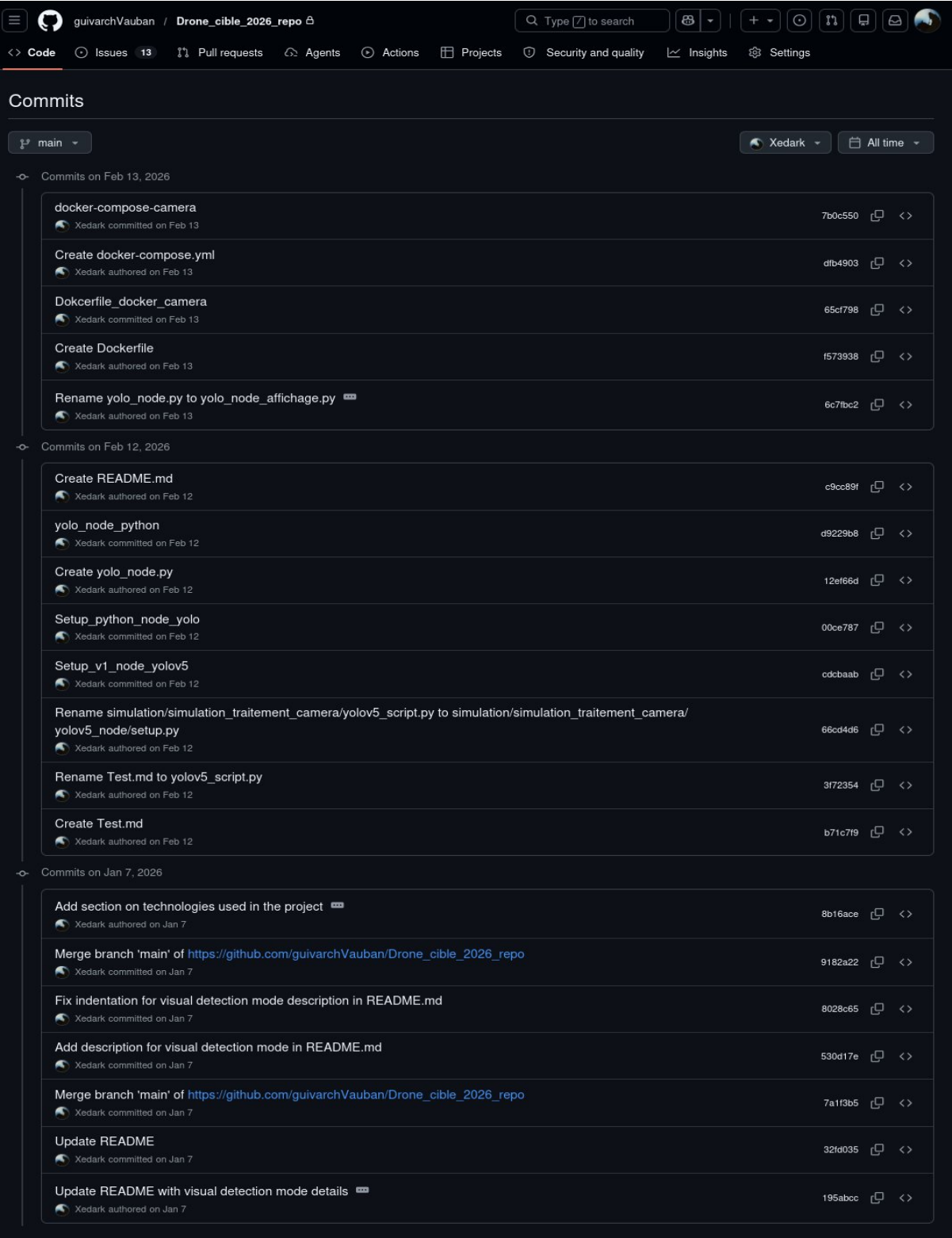
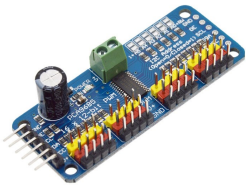


Figure 3 — Historique des commits sur la branche main

Annexe D – Équipement utilisés

Composant	Rôle
Raspberry Pi 5 	Unité centrale : ROS 2, YOLOv5, navigation
Webcam USB 	Capture du flux vidéo pour la détection de cibles
Antennes Wi-Fi ×2 	Retour vidéo longue portée vers le poste opérateur
PCA9685 	Contrôleur PWM 12 bits (I2C) pour moteurs et gouvernail
L298N 	Pont en H : contrôle vitesse/sens + isolation électrique
Moteurs DC ×2	Propulsion via commande linear.x

	
Servomoteur 	Gouvernail via commande angular.z